PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

Project Title: *LHDN Fraud Detective Agent*

Prepared By: *Team CacheMe (Domain 1 – Group ID: 103)*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: InvoiceOS - AI Agentic e-Invoicing System

Version: 3.0

1. Project Overview

- **Problem Statement:** Malaysian invoice review is still fragmented across PDFs, images, XML, JSON, Word files, Excel files, and manually keyed records. Finance and procurement operators must extract fields, verify the supplier, check for duplicate claims, compare billed line items against agreed pricing, decide whether the document is an enterprise invoice or a retail receipt, and maintain an audit trail. The current manual path is slow, repetitive, and vulnerable to missed fraud signals.

- **Target Domain:** AI Systems & Agentic Workflow Automation. The product uses GLM as the core reasoning engine to transform unstructured invoice artifacts into structured JSON that a deterministic verification workflow can act on.

- **Proposed Solution Summary:** The LHDN Fraud Detective Agent is a web-based verification workflow. A user uploads a document or types an operator command, the backend preprocesses the file, calls the GLM extraction model, requests human correction for missing critical fields when necessary, checks the extracted result against the vendor and contract database, and then routes the document into one of four outcomes: approved B2B invoice, pending review, fraud detected, or cash receipt pending consolidation. Approved B2B invoices generate a printable certificate; B2C receipts are accumulated for month-end consolidation.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Background & Business Objective

- **Background of the Problem:** The repository reflects a workflow where invoice evidence may arrive in multiple file formats and quality levels. Without automation, the operator has to normalize the input, interpret layout and OCR noise, manually cross-check supplier identity, inspect invoice uniqueness, and compare unit prices against contractual baselines. Retail receipts create an additional burden because they should be aggregated rather than processed one by one as enterprise invoices.

- **Importance of Solving This Issue:** A working invoice-clearing workflow reduces review friction, shortens approval cycles, improves fraud awareness, and creates a consistent audit trail across B2B approvals and B2C consolidation. In the context of the hackathon domain, it also demonstrates that AI is doing more than chat generation: it is coordinating the first mile of document understanding and triggering the next workflow stage.

- **Strategic Fit / Impact:** The solution directly satisfies the domain brief by combining unstructured-input understanding, multi-step reasoning, workflow orchestration, and structured actionable outputs. It also addresses multiple judging areas at once: product relevance, originality, MVP prioritization, workflow integration, and clear system logic.

3. Product Purpose

3.1. Main Goal of the System: To turn messy invoice and receipt artifacts into operator-ready compliance decisions with minimal manual intervention and a clear audit trail.

3.2. Intended Users (Target Audience):

- Accounts payable or finance operations staff who need a faster way to validate incoming supplier invoices.
- Procurement and compliance reviewers who need vendor, TIN, duplicate, and pricing checks before approval.
- Retail operations staff who need daily cash receipts logged and then consolidated into a compliant month-end batch.

© UMHackathon 2026

4. System Functionalities

4.1. Description: The system behaves as an AI-assisted invoice clearing workflow. It does not stop at extraction. The extracted JSON becomes the input for a deterministic decision engine that verifies supplier identity, checks invoice uniqueness, checks contract pricing, classifies B2B versus B2C, records the resulting workflow state, and optionally produces a printable certificate.

4.2. Key Functionalities:

- Multi-format intake: accepts PNG, JPEG, XML, JSON, PDF, DOCX, and XLSX documents from a single chat-like interface.
- AI-powered extraction: sends either a base64 image payload or document-derived raw text to the GLM model and expects a fixed JSON schema.
- Human-in-the-loop correction: if the invoice number or supplier name is missing, the UI moves into a guided correction state and resubmits completed JSON.
- Fraud and compliance checks: validates supplier registration, TIN match, duplicate invoice history, and overbilling against SKU-level contract prices.
- Outcome routing: returns approved, pending review, fraud detected, or cash receipt pending consolidation based on the extracted and verified record.
- Retail month-end consolidation: aggregates pending B2C receipts into a single approved ledger record with a generated UUID.
- Certificate generation: approved B2B invoices include printable HTML certificate content from the backend.

© UMHackathon 2026

- Voice-assisted operator input: the frontend supports microphone-based instruction entry in browsers that expose *SpeechRecognition* or *webkitSpeechRecognition*.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: The repository calls the ILMU chat-completions endpoint with model name ilmu-glm-5.1. This is the only model integrated in the current codebase. The choice is appropriate for the prototype because the same model handles text-only extraction and image-assisted extraction while returning structured JSON used by downstream workflow logic.

4.3.2. Prompting Strategy: The current implementation uses a single-call schema-constrained extraction strategy. A fixed system prompt defines the target JSON contract and OCR heuristics. For images, the backend adds a visual-layout guidance prompt. For text-derived documents, the backend appends the extracted raw content after a separator. Multi-step workflow behavior then continues in Python rather than chaining multiple LLM calls.

Input Source	Preprocessing	Model Context Sent
PNG / JPEG	Convert file bytes to base64	User text + visual extraction rules + image_url payload
XML / text	Decode as UTF-8 text	User text + raw invoice data block
JSON	Parse and pretty-print JSON	User text + raw invoice data block
DOCX	Extract non-empty paragraphs and tables	User text + raw invoice data block

© UMHackathon 2026

XLSX	Read with pandas and serialize to TSV-like text	User text + raw invoice data block
PDF	Extract page text with PyPDF2	User text + raw invoice data block

4.3.3. Context & Input Handling: The prototype currently sends the full extracted content to the model. There is no explicit truncation, chunking, summarization, or application-level token guard in the current code. This keeps the design simple for a hackathon MVP, but oversized documents remain an identified risk that should be hardened in the next iteration.

4.3.4. Fallback & Failure Behavior: The GLM call uses a 120-second timeout and allows one retry on timeout or certain gateway failures. If the model output does not contain valid JSON, the backend returns an error state instead of attempting to continue. If the JSON is present but critical fields are missing, the system degrades gracefully into a human-correction workflow. If supplier or SKU checks fail, the system returns warning or fraud states rather than silently approving the invoice.

5. User Stories & Use Cases

- **User Stories:**
  - As a finance operator, I want to upload an invoice in whatever format I received it so that I do not need to manually retype supplier and invoice data.
  - As a compliance reviewer, I want the system to flag duplicate invoice numbers and mismatched TINs so that suspicious claims are surfaced early.
  - As a procurement officer, I want the system to compare SKU unit prices against contract prices so that overbilling is caught immediately.
  - As a retail administrator, I want daily cash receipts logged separately and consolidated at month end so that B2C records follow a cleaner operational workflow.

© UMHackathon 2026

- As a finance operator, I want missing fields handled through guided follow-up prompts so that the workflow can recover from imperfect OCR instead of failing outright.

- **Use Cases (Main Interactions):**

Use Case 1: Unstructured Data Processing Flow

This use case describes the primary interaction where users submit unstructured data for conversion into e-Invoices.

Flow:

1. User uploads or submits unstructured data (PDF, image, or text input).
2. The system ingests and aggregates the data into a centralized platform.
3. The AI model extracts relevant information and maps it to required fields.
4. The system validates the extracted data against LHDN requirements.
5. Any missing or incorrect fields are flagged in the dashboard.

Use Case 2: Error Detection and User Correction Flow

This use case outlines how users interact with the system when issues are detected.

Flow:

1. The system identifies missing or inconsistent fields.
2. The dashboard highlights problematic fields for each document.
3. The user receives a notification via the system inbox.
4. The user edits the fields manually or uploads a revised document.
5. The system reprocesses and revalidates the updated data.

Use Case 3: e-Invoice Generation Flow

This use case describes the process of generating structured e-Invoices after validation.

Flow:

© UMHackathon 2026

1. The system confirms that all required fields are complete and valid.
2. The data is transformed into a structured format compliant with LHDN.
3. A finalized e-Invoice is generated and stored within the system.

Use Case 4: Accountant Review and Approval Flow

This use case represents the second layer of verification in the workflow.

Flow:

1. The accountant accesses the dashboard to review generated e-Invoices.
2. The system presents extracted data, validation results, and flagged uncertainties.
3. The accountant evaluates the invoice for correctness and compliance.
4. If the invoice is accurate, the accountant approves and digitally acknowledges it.
5. If issues are identified, the accountant either edits the fields directly or requests additional information from the business user.
6. The updated invoice is resubmitted for accountant review, forming an iterative approval loop.

Use Case 5: Submission to LHDN Flow

This use case describes the final stage of the workflow.

Flow:

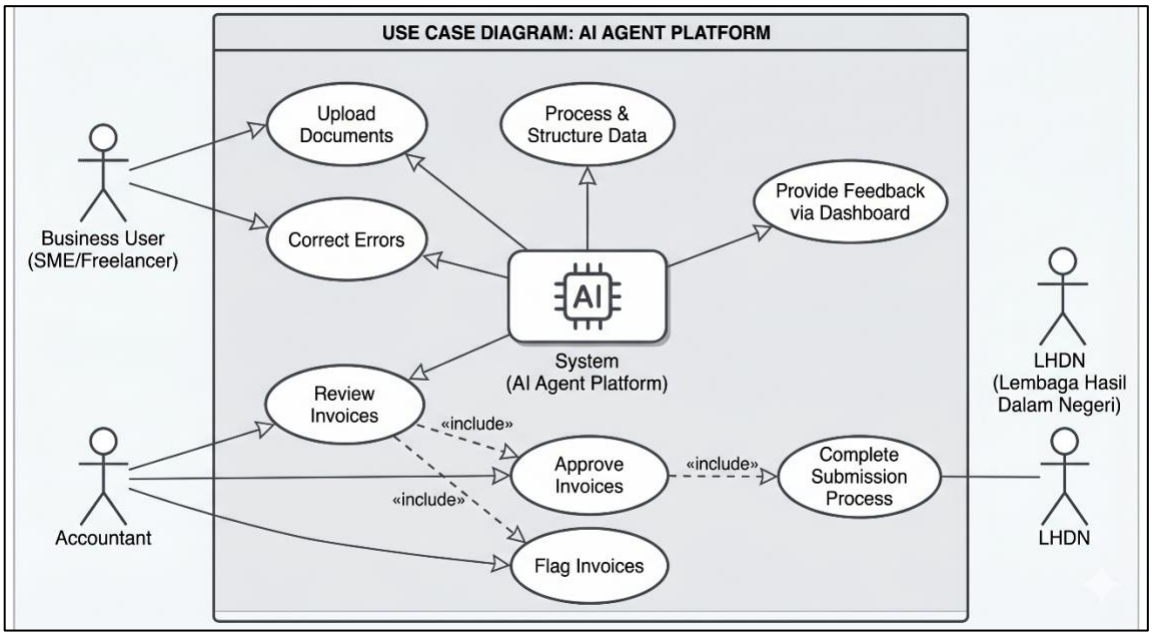
1. Approved invoices are passed to the submission module.
2. The system submits the invoice to LHDN via a simulated (mock) integration.
3. The system receives and records the submission response.
4. The invoice status is updated in the dashboard.

- **Use Case Diagram:** The system can be represented through a use case diagram involving three primary actors:

© UMHackathon 2026

- Business User (SME/Freelancer)
- Accountant
- System (AI Agent Platform)

The Business User interacts with the system to upload documents and correct errors. The system processes and structures the data while providing feedback through the dashboard. The Accountant interacts with the system to review, approve, or flag invoices. Finally, the system communicates with LHDN (via a mock interface) to complete the submission process.



6. Features Included (Scope Definition)

- A single-page operator interface for file upload, text input, voice input, and workflow responses.
- GLM-based invoice extraction with a fixed JSON schema for supplier, buyer, metadata, financials, and line items.

© UMHackathon 2026
Email: umhackathon@um.edu.my

- Guided missing-field repair for the two critical fields enforced by the current code: Invoice Number and Supplier Name.
- SQLite-backed vendor verification, duplicate detection, contract-price checks, and audit ledger writes.
- B2B approval flow with printable certificate generation.
- B2C receipt logging and month-end consolidation command.

7. Features Not Included (Scope Control)

- Real LHDN network integration. The current implementation generates UUIDs and stores approved records locally but does not call a separate LHDN API.
- Authentication, role-based access control, or multi-user permission boundaries.
- Cloud deployment automation, containerization, or distributed microservices.
- Automated prompt chunking, context compression, or token-budget governance.
- Advanced anomaly models beyond the current deterministic vendor, duplicate, and unit-price checks.
- Formal observability dashboards, alerting pipelines, and production-grade secret management.

8. Assumptions & Constraints

Category	Current Constraint or Assumption
LLM Cost Constraint	The prototype does not expose token accounting, so cost control is qualitative rather than measured. Cost is currently limited by using one extraction call per document and doing the rest of the decision logic in Python.
Technical Constraint	The runnable stack is a monolithic FastAPI app plus a static frontend backed by SQLite. The live seeded SQLite setup contains 4

© UMHackathon 2026

	vendors, 5 products, 5 contract rules, and 3 historical processed invoices.
Reference Data Constraint	The repository also contains a larger SQL reference dump, but the executable setup script loads only the smaller SQLite dataset. Detection quality is therefore bounded by the loaded data.
Performance Constraint	The GLM request is synchronous, uses a 120-second timeout, and has no background job queue or explicit chunking layer.
User Input Constraint	The workflow assumes a human operator is available to repair missing fields and review warning / fraud outcomes.
Browser Constraint	Voice input depends on browser support for *SpeechRecognition* or *webkitSpeechRecognition* and is currently configured for *en-US*.

9. Risks & Questions Throughout Development

- How should the product enforce input-size limits before a very large PDF or spreadsheet reaches the model?

- Should approved B2B invoices also be written to processed_invoices so duplicate detection covers new approvals and not only the seeded historical ledger?

- How should the product prevent duplicate retail receipt insertion when the same receipt is uploaded repeatedly?

© UMHackathon 2026
Email: umhackathon@um.edu.my

- What is the minimum vendor and contract dataset size required before warning rates become operationally acceptable outside a demo environment?

- How should secrets, CORS policy, and real LHDN integration be hardened before any external deployment?

- What additional business rules should be introduced for subtotal / tax cross-checking, buyer validation, and supplier status enforcement?

© UMHackathon 2026

Email: umhackathon@um.edu.my